

END-TERM PROJECT-02 REPORT

ON

Flappy Bird Game

B.Tech. [Computer Science and Engineering]

Semester: 6th

Section: 23412K1

Course Title & Code: Game Programming with C++ (CSE 3545)

Submitted By:

Student Name: Amit Kumar

Registration No: 2341011129

Guided By: Ishan Ayus

Academic Year: 2025 - 26



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING AND TECHNOLOGY, INSTITUTE OF
TECHNICAL EDUCATION AND RESEARCH
SIKSHA 'O' ANUSANDHAN (DEEMED TO BE) UNIVERSITY
BHUBANESWAR, ODISHA, INDIA

ABSTRACT

This project focuses on the design and development of a simple Flappy Bird-style game implemented using C++ and the SFML (Simple and Fast Multimedia Library). The main objective of this project is to create an interactive 2D game that demonstrates real-time rendering, event handling, and object-oriented programming concepts.

The game allows the player to control a bird that must navigate through moving pipes without colliding. The score increases as the player successfully passes obstacles, and a high score is maintained using file handling. This project enhances understanding of game loops, collision detection, graphics rendering, and user interaction.

Submitted By

Amit Kumar

Guided By

Ishan Ayus

TABLE OF CONTENTS

Section No.	Topic	Page No.
01	Introduction	1
02	Problem Statement	2
03	Objectives of the Project	3
04	Tools and Technologies Used	4
05	Methodology	5
06	Results Analysis and Screenshots	6-9
07	Logical Understanding	10
08	Conclusion	12
09	Future Scopes and Application	13
	References	14
	Annexure	15-27

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Home Screen / Start State	6
Figure 2	Playing State	7
Figure 3	Game Over State	7
Figure 4	Workflow Diagram	11

01. Introduction

C++ is a powerful programming language widely used in system programming, application development, and game development. It supports object-oriented programming, which allows efficient structuring of complex applications.

This project involves the development of a 2D game inspired by Flappy Bird using C++ and SFML. The game provides real-time interaction and visual feedback, making it a practical example of multimedia programming.

The main aim of this project is to apply programming concepts such as classes, file handling, loops, and event-driven logic into a real-world application.

02. Problem Statement

The problem addressed in this project is to design and implement an interactive 2D game where:

- A bird continuously falls due to gravity.
- The player must control the bird's movement to avoid obstacles (pipes).
- The game ends when a collision occurs.
- The system tracks and stores the highest score achieved.

The challenge lies in implementing smooth gameplay, accurate collision detection, and persistent score storage.

03. Objectives of the Project

The objectives of this project are defined to provide a clear direction for the design and development of the Flappy Bird Game. The objectives of this practical project are:

- To develop a graphical game using C++ and SFML.
- To implement object-oriented concepts such as classes and encapsulation.
- To simulate real-time motion using game loops.
- To implement collision detection logic.
- To maintain and update high scores using file handling.
- To enhance debugging and problem-solving skills.

To gain practical exposure to real-time programming applications.

04. Tools and Technologies Used

The following tools and technologies were used in this project:

- Programming Language: C++/ SFML
- Compiler: GCC / Turbo C / Dev-C++
- Development Environment: Text Editor / VS Code
- Operating System: Windows / Linux

These tools provide a suitable environment for writing, compiling, and executing C programs.

05. Methodology

The methodology followed during the development of this project includes:

1. Understanding the Problem:

Identifying the requirements and expected output. This step helps in clearly defining the game rules and overall functionality before implementation begins.

2. Algorithm Design:

Creating a step-by-step logical solution. It ensures proper flow of the game including movement, scoring, and collision handling.

3. Program Development:

Writing the code in C++ language using SFML. The program is divided into multiple classes to maintain modularity and readability.

4. Compilation and Execution:

Running the program using a compiler. Errors are identified during compilation and corrected for smooth execution.

5. Testing and Debugging:

Checking for errors and improving accuracy. Multiple test cases are considered to ensure stable gameplay and correct scoring.

6. Final Output:

Ensuring the program produces correct results. The final game is verified for smooth performance and user interaction.

06. Results Analysis and Screenshots

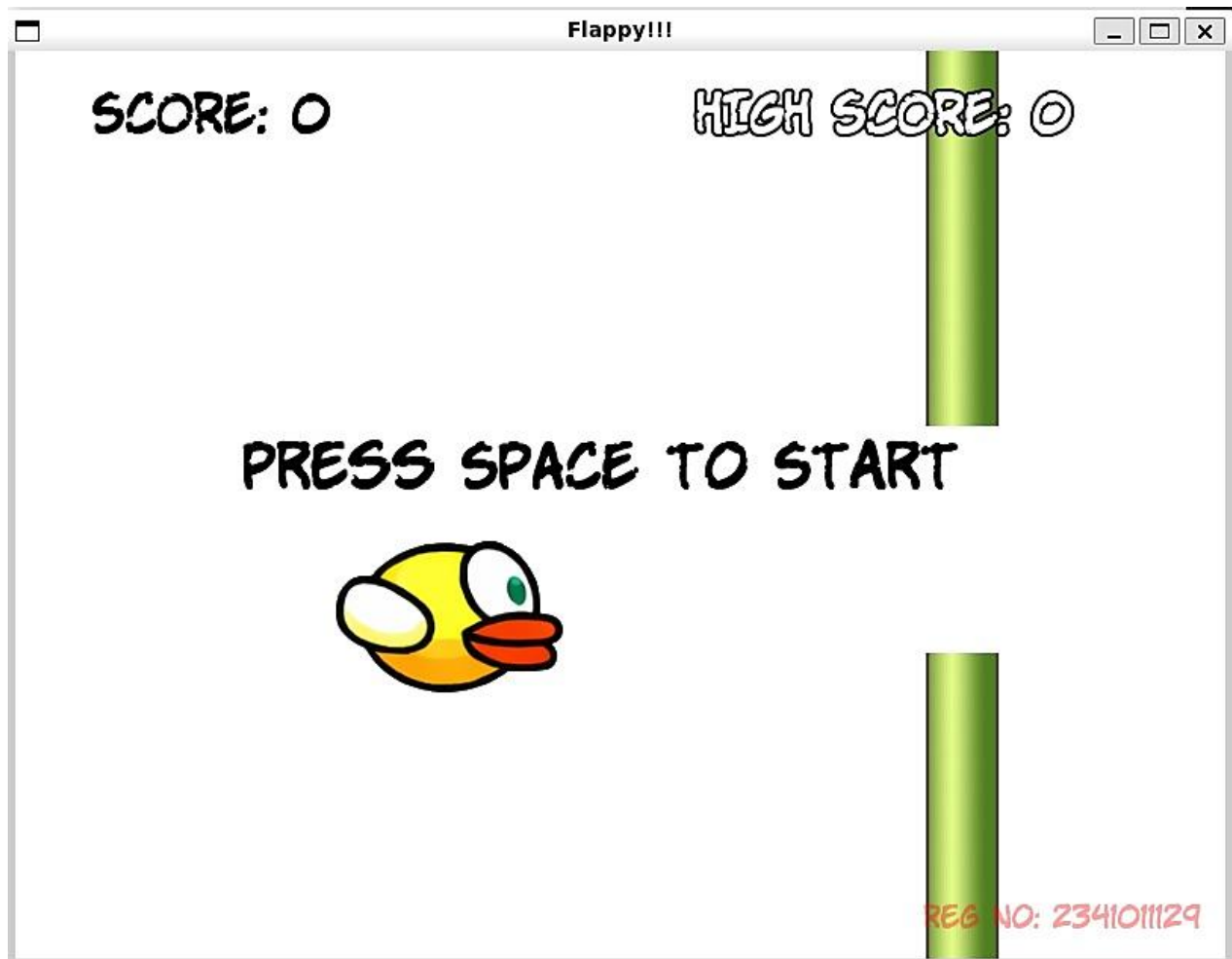
This section describes the results obtained after implementing the C++ program.

The program was successfully executed and produced the expected output. It demonstrates the correct use of programming concepts and provides accurate results based on user input.

- Smooth bird movement with gravity simulation
- Pipes generated dynamically
- Collision detection works correctly
- Score increments properly
- High score persists across sessions using file

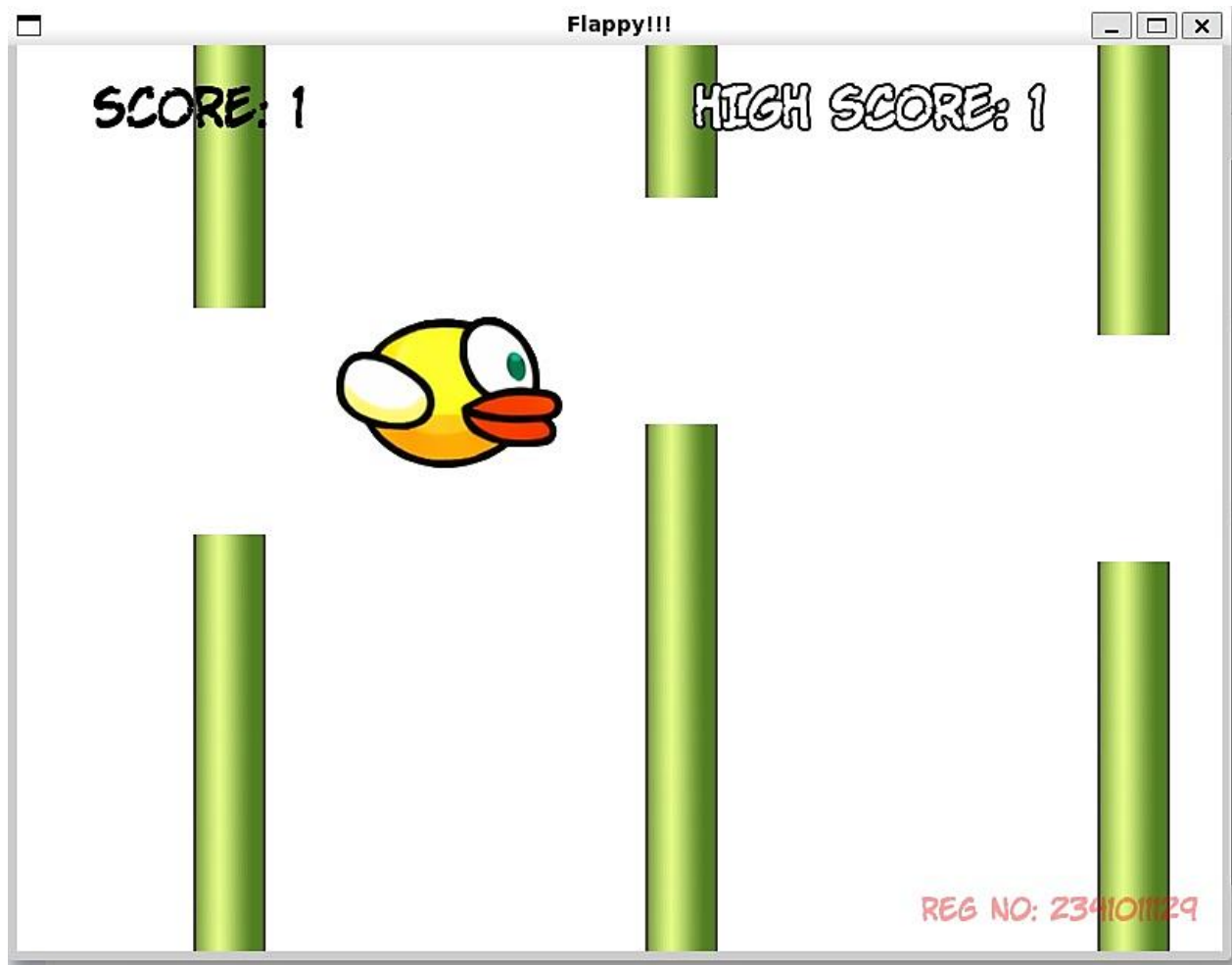
Output Screenshots:

Figure 1 (Home Screen / Start State):



The image shows the start screen of the Flappy Bird Game. The bird is positioned in the center, and the message "PRESS SPACE TO START" is displayed. The score and high score are shown at the top of the screen.

Figure 2 (Playing State):



The image shows the gameplay screen where the bird moves through the pipes. The player controls the bird to avoid collisions while the score increases as pipes are crossed.

Figure 3 (Game Over State):



The image shows the game over screen after the bird collides with a pipe. The message "GAME OVER! PRESS SPACE TO RESTART" is displayed, allowing the player to restart the game.

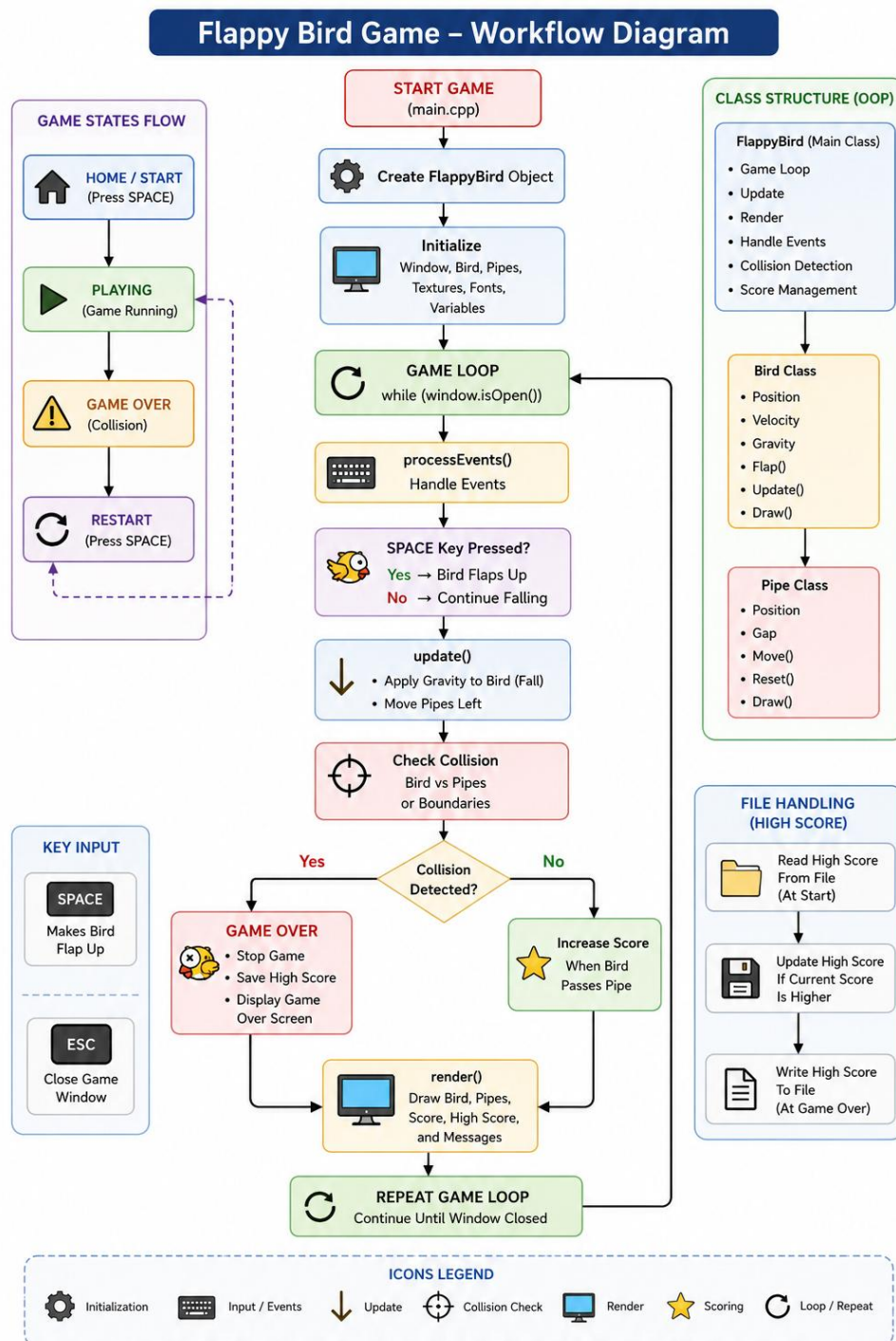
07. Logical Understanding

The logic of the program is based on:

- Continuous game loop execution
- Gravity affecting bird position
- Player input (space key) to move bird upward
- Pipes moving horizontally across screen
- Collision detection using bounding boxes
- Score increment when bird passes pipes
- File handling to store and retrieve high score

This project helps in understanding the flow of control and logical execution in C programming. This program also demonstrates real-time system behaviour and event-driven programming.

Figure 4: Workflow Diagram



The workflow diagram shows the complete working of the Flappy Bird Game developed using C++ and SFML. The game initializes the bird, pipes, textures, and game window. The player presses the space key to make the bird flap upward while gravity continuously pulls it downward. Pipes move horizontally across the screen, and collision detection checks if the bird hits a pipe or screen boundary. The score increases when the bird successfully passes pipes, and the high score is saved using file handling. If a collision occurs, the game enters the Game Over state.

08. Conclusion

This project was successfully completed using C++ and SFML. It provided practical exposure to game development concepts such as rendering, event handling, and object-oriented design.

The implementation strengthened understanding of:

- Game loops
- Physics simulation
- File handling
- Modular programming

Overall, this project significantly improved programming skills and logical thinking.

This project provides valuable experience in developing simple applications and improves overall programming confidence.

09. Future Scopes and Application

The future improvements and applications of this project include:

- Adding sound effects and background music
- Improving graphics and animations
- Adding difficulty levels
- Implementing menus (start/pause/restart)
- Using advanced physics for realistic motion
- Expanding into mobile or web-based versions

This project can be extended into full-scale game development or used as a base for learning advanced graphics programming.

References

- (1) Practical C Programming by B.M. Harwani (Packt)

Annexure

The following section includes the code of the game including the explanation for different portions.

Bird.cpp

```
#include "Bird.h"
```

```
Bird::Bird()
```

```
{  
    m_Gravity = 1500.0f;  
    m_FlapStrength = 500.0f;  
    reset();  
}
```

```
void Bird::setTexture(const Texture &texture)
```

```
{  
    m_Shape.setTexture(texture);  
    m_Shape.setScale(0.03f, 0.03f);  
}
```

```
void Bird::update(Time dt)
```

```
{  
    m_VelocityY += m_Gravity * dt.asSeconds();  
    m_Position.y += m_VelocityY * dt.asSeconds();  
    m_Shape.setPosition(m_Position);  
}
```

```
void Bird::flap()
```

```
{  
    m_VelocityY = -m_FlapStrength;  
}
```

```
FloatRect Bird::getBounds() const
{
    return m_Shape.getGlobalBounds();
}
```

```
Vector2f Bird::getPosition() const
{
    return m_Position;
}
```

```
void Bird::reset()
{
    m_Position.x = 200.f;
    m_Position.y = 300.f;
    m_VelocityY = 0.f;
    m_Shape.setPosition(m_Position);
}
```

Bird.h

```
#pragma once
#include <SFML/Graphics.hpp>
```

```
using namespace sf;
```

```
class Bird{
private:
    Sprite m_Shape;
    Vector2f m_Position;
    float m_VelocityY;
```

```
float m_Gravity;  
float m_FlapStrength;
```

```
public:  
    Bird();  
    void setTexture(const Texture &texture);  
    void update(Time dt);  
    void flap();  
    FloatRect getBounds() const;  
    Vector2f getPosition() const;  
    void reset();  
};
```

Pipe.cpp

```
#include "Pipe.h"
```

```
Pipe::Pipe() : m_IsPassed(false), m_IsActive(false) {}
```

```
void Pipe::spawn(float startX, float gapY, float gapHeight, float speed, const Texture  
&texture)
```

```
{  
    m_X = startX;  
    m_Speed = speed;  
    m_IsPassed = false;  
    m_IsActive = true;  
  
    m_TopPipe.setTexture(texture);  
    m_BottomPipe.setTexture(texture);  
  
    m_TopPipe.setOrigin(0.f, 0.f);  
    m_TopPipe.setPosition(m_X, gapY - (gapHeight / 2.0f));
```

```

    m_TopPipe.setScale(1.f, -2.5f);

    m_BottomPipe.setOrigin(0.f, 0.f);
    m_BottomPipe.setPosition(m_X, gapY + (gapHeight / 2.0f));
    m_BottomPipe.setScale(1.f, 2.5f);
}

void Pipe::update(Time dt)
{
    if (!m_IsActive)
        return;

    m_X -= m_Speed * dt.asSeconds();
    m_TopPipe.setPosition(m_X, m_TopPipe.getPosition().y);
    m_BottomPipe.setPosition(m_X, m_BottomPipe.getPosition().y);
}

bool Pipe::checkCollision(FloatRect birdBounds) const
{
    if (!m_IsActive)
        return false;

    if (m_TopPipe.getGlobalBounds().intersects(birdBounds) ||
        m_BottomPipe.getGlobalBounds().intersects(birdBounds)){
        return true;
    }
    return false;
}

void Pipe::draw(RenderWindow &window){
    if (!m_IsActive)

```

```

        return;
    window.draw(m_TopPipe);
    window.draw(m_BottomPipe);
}

float Pipe::getX() const { return m_X; }
bool Pipe::isPassed() const { return m_IsPassed; }
void Pipe::setPassed() { m_IsPassed = true; }

```

Pipe.h

```

#pragma once
#include <SFML/Graphics.hpp>
using namespace sf;

class Pipe{
private:
    Sprite m_TopPipe;
    Sprite m_BottomPipe;
    float m_X;
    float m_Speed;
    bool m_IsPassed;
    bool m_IsActive;

public:
    Pipe();
    void spawn(float startX, float gapY, float gapHeight, float speed, const Texture &texture);
    void update(Time dt);
    bool checkCollision(FloatRect birdBounds) const;
    void draw(RenderWindow &window);

```

```

float getX() const;
bool isPassed() const;
void setPassed();
};

```

FlappyBird.cpp

```

#include "FlappyBird.h"
#include <sstream>
#include <cstdlib>

```

```

FlappyBird::FlappyBird() : window(VideoMode(800, 600), "Flappy!!!"),
                           score(0), highScore(0), gameSpeed(250.0f),
                           isGameOver(false), isGameStarted(false)
{
    window.setFramerateLimit(60);

    loadHighScore();

    font.loadFromFile("/mnt/g/wsl/GPWC/Flappy/Font/KOMIKAP_.ttf");

    birdTexture.loadFromFile("/mnt/g/wsl/GPWC/Flappy/Graphics/Bird.png");
    player.setTexture(birdTexture);

    pipeTexture.loadFromFile("/mnt/g/wsl/GPWC/Flappy/Graphics/Pipes.png");

    scoreText.setFont(font);
    scoreText.setCharacterSize(35);
    scoreText.setFillColor(Color::Black);
    scoreText.setPosition(50, 20);

```



```
highScoreText.setFont(font);
highScoreText.setCharacterSize(35);
highScoreText.setFillColor(Color::White);
highScoreText.setOutlineColor(Color::Black);
highScoreText.setOutlineThickness(2.0f);
highScoreText.setPosition(450, 20);
```

```
messageText.setFont(font);
messageText.setCharacterSize(40);
messageText.setFillColor(Color::Black);
messageText.setPosition(150, 250);
messageText.setString("Press SPACE to Start");
```

```
watermarkText.setFont(font);
watermarkText.setString("Reg No: 2341011129");
watermarkText.setCharacterSize(20);
watermarkText.setFillColor(Color(255, 0, 0, 100));
watermarkText.setPosition(600.f, 560.f);
```

```
resetGame();
```

```
}
```

```
void FlappyBird::loadHighScore(){
    std::ifstream file("assets/highscore.txt");
    if (file.is_open()){
        file >> highScore;
        file.close();
    }
    else{
        highScore = 0;
    }
}
```

```
}
```

```
void FlappyBird::saveHighScore(){  
    std::ofstream file("assets/highscore.txt");  
    if (file.is_open())  
    {  
        file << highScore;  
        file.close();  
    }  
}
```

```
void FlappyBird::run(){  
    Clock clock;  
    while (window.isOpen()){  
        processEvents();  
  
        Time dt = clock.restart();  
        if (isGameStarted && !isGameOver){  
            update(dt);  
        }  
        render();  
    }  
}
```

```
void FlappyBird::processEvents(){  
    Event event;  
    while (window.pollEvent(event)){  
        if (event.type == Event::Closed)  
            window.close();  
  
        if (event.type == Event::KeyPressed && event.key.code == Keyboard::Space){
```

```

        if (!isGameStarted){
            isGameStarted = true;
            player.flap();
        }
        else if (isGameOver){
            resetGame();
            isGameStarted = true;
            player.flap();
        }
        else
            player.flap();
    }
}
}

```

```

void FlappyBird::update(Time dt){
    player.update(dt);

```

```

    FloatRect birdBounds = player.getBounds();

```

```

    if (birdBounds.top < 0 || birdBounds.top + birdBounds.height > 600)
        isGameOver = true;

```

```

    for (int i = 0; i < MAX_PIPES; i++){
        pipes[i].update(dt);

```

```

        if (pipes[i].checkCollision(birdBounds))
            isGameOver = true;

```

```

        if (!pipes[i].isPassed() && pipes[i].getX() + 50.f < player.getPosition().x){
            pipes[i].setPassed();

```

```

        score++;
    }

    if (pipes[i].getX() < -100.f){
        float highestX = 0;
        for (int j = 0; j < MAX_PIPES; j++){
            if (pipes[j].getX() > highestX)
                highestX = pipes[j].getX();
        }
        float newGapY = 150.f + (rand() % 300);
        pipes[i].spawn(highestX + 300.f, newGapY, 150.f, gameSpeed, pipeTexture);
    }
}

if (isGameOver){
    if (score > highScore){
        highScore = score;
        saveHighScore();
    }
    messageText.setString("GAME OVER!\nPress SPACE to Restart");
}

scoreText.setString("SCORE: " + std::to_string(score));
highScoreText.setString("HIGH SCORE: " + std::to_string(highScore));
}

void FlappyBird::resetGame(){
    score = 0;
    isGameOver = false;
    isGameStarted = false;
    player.reset();
}

```

```

messageText.setString("Press SPACE to Start");

float startX = 600.f;
for (int i = 0; i < MAX_PIPES; i++){
    float randomGapY = 150.f + (rand() % 300);
    pipes[i].spawn(startX + (i * 300.f), randomGapY, 150.f, gameSpeed, pipeTexture);
}
scoreText.setString("SCORE: 0");
highScoreText.setString("HIGH SCORE: " + std::to_string(highScore));
}

void FlappyBird::render(){
    window.clear(Color::White);
    for (int i = 0; i < MAX_PIPES; i++){
        pipes[i].draw(window);
    }

    Sprite birdSprite;
    birdSprite.setTexture(birdTexture);
    birdSprite.setPosition(player.getPosition());
    window.draw(birdSprite);

    window.draw(scoreText);
    window.draw(highScoreText);

    if (!isGameStarted || isGameOver)
        window.draw(messageText);

    window.draw(watermarkText);
    window.display();

```

```
}
```

FlappyBird.h

```
#pragma once
```

```
#include <SFML/Graphics.hpp>
```

```
#include <fstream>
```

```
#include <iostream>
```

```
#include "Bird.h"
```

```
#include "Pipe.h"
```

```
using namespace sf;
```

```
class FlappyBird{
```

```
private:
```

```
    RenderWindow window;
```

```
    Texture birdTexture;
```

```
    Texture pipeTexture;
```

```
    Bird player;
```

```
    static const int MAX_PIPES = 5;
```

```
    Pipe pipes[MAX_PIPES];
```

```
    Font font;
```

```
    Text scoreText;
```

```
    Text highScoreText;
```

```
    Text messageText;
```

```
    Text watermarkText;
```

```

int score;
int highScore;
float gameSpeed;
bool isGameOver;
bool isGameStarted;

void loadHighScore();
void saveHighScore();
void processEvents();
void update(Time dt);
void render();
void resetGame();

public:
    FlappyBird();
    void run();
};

```

Main.cpp

```

#include "FlappyBird.h"
#include <ctime>
#include <cstdlib>

int main(){
    srand(static_cast<unsigned>(time(nullptr)));

    FlappyBird game;
    game.run();

    return 0;
}

```